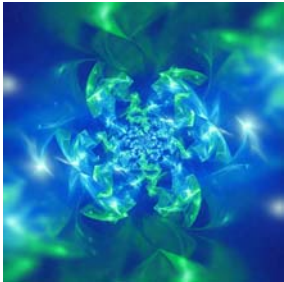


## B 2.4.4 Recursion Python (HL Only)



B 2.4.4 Explain the fundamental concept of recursion and its applications in programming. (HL only)

Unlock the power of recursion! This section explores the fascinating world of recursive programming. Learn how functions can call themselves to solve problems, discover the advantages and limitations of this technique, and see how it's applied in real-world scenarios.

### Introduction

Recursion is a powerful and elegant programming technique that allows you to solve problems by breaking them down into smaller, self-similar subproblems. In this chapter, we'll explore the fundamentals of recursion, its advantages, disadvantages, and applications in Java.

### What is Recursion?

Recursion is a technique where a function calls itself to solve a problem. It's like a set of Russian nesting dolls, each containing a smaller version of itself. The key components of a recursive function are:

**Base Case:** The condition that stops the recursion. It's like the smallest doll that doesn't contain any others. Without a base case, the recursion would continue infinitely, leading to a stack overflow error.

**Recursive Case:** The step where the function calls itself with a smaller subproblem. It's like opening a doll to reveal a smaller one. The recursive case must eventually lead to the base case to ensure the recursion terminates.

### Simple Example: Factorial

Let's illustrate recursion with a classic example: calculating the factorial of a number. The factorial of a non-negative integer  $n$ , denoted by  $n!$ , is the product of all positive integers less than or equal to  $n$ .

to n. For example,  $5! = 5 * 4 * 3 * 2 * 1 = 120$ .

Here's the recursive Python code to calculate the factorial:

```
def factorial(n):  
    if n == 0: # Base case: 0! = 1  
        return 1  
    else: # Recursive case: n! = n * (n-1)!  
        return n * factorial(n - 1)
```

Let's trace the execution of factorial(4):

factorial(4) = 4 \* factorial(3)

factorial(3) = 3 \* factorial(2)

factorial(2) = 2 \* factorial(1)

factorial(1) = 1 \* factorial(0)

factorial(0) = 1 (base case)

Now, the values are returned back up the chain: factorial(1) = 1 \* 1 = 1 factorial(2) = 2 \* 1 = 2  
factorial(3) = 3 \* 2 = 6 factorial(4) = 4 \* 6 = 24

## Advantages of Recursion

**Code Readability:** Recursion can make code more concise and easier to understand, especially for problems with repetitive structures. The recursive solution often closely mirrors the problem's definition, making the code more intuitive.

**Elegance:** Recursion can provide elegant solutions for problems with inherent recursive structures, such as tree traversal, fractal generation, and divide-and-conquer algorithms.

## Disadvantages of Recursion

**Stack Overflow:** Each recursive call adds a new frame to the call stack. Excessive recursion can exhaust the stack memory, leading to stack overflow errors and crashing the program.

**Performance:** Recursive function calls can have overhead in terms of time and memory compared to iterative solutions. This overhead can be significant for deep recursion or problems with simple recursive structures that can be easily converted to loops.

# Applications of Recursion

**Quicksort:** A divide-and-conquer sorting algorithm that uses recursion to partition the data around a pivot and sort the subpartitions.

## How Quicksort works

### Quicksort: A Recursive Divide-and-Conquer Algorithm

Quicksort is an efficient sorting algorithm that uses a **divide-and-conquer** strategy, making it naturally suited for a recursive implementation

#### Key Steps in Quicksort

The Quicksort algorithm involves three main steps, two of which are executed recursively:

1. **Choose a Pivot:** Select an element from the list, often called the **pivot**. This element will help partition the list.
2. **Partition:** Rearrange the elements in the list so that all elements **less than or equal to** the pivot are to its left, and all elements **greater than** the pivot are to its right. After this step, the pivot is in its **final sorted position**.
3. **Recursively Sort Sub-lists:** Recursively apply the quicksort algorithm to the sub-list of elements to the left of the pivot and the sub-list to the right of the pivot.

The **base case** (the condition that stops the recursion) is reached when a sub-list contains zero or one element, as such a list is already sorted

#### Step-by-Step Example

Let's trace the quicksort process on a small array: [4, 3, 1, 2, 5, 9, 7, 10, 6].

Initial Array:

[4, 3, 1, 2, 5, 9, 7, 10, 6]

##### 1. First Partition (Divide)

- **Choose Pivot:** We select the element **5** as the pivot.
- **Partition:** We rearrange the array so elements  $\leq 5$  are on the left and elements  $\geq 5$  are on the right.
  - The numbers 4, 3, 1, 2 are less than 5.
  - The numbers 9, 7, 10, 6 are greater than 5.

The array is now divided into two sub-problems, with the pivot **5** correctly placed:

[4, 3, 1, 2]    5    [9, 7, 10, 6]

(Sub-list 1) (Pivot) (Sub-list 2)

[4, 3, 1, 2]    5    [9, 7, 10, 6]

## 2. Recursive Call on Sub-list 1: [4, 3, 1, 2]

- **Choose Pivot:** We select **2** as the pivot for this sub-list.
- **Partition:** Elements  $\leq 2$  on the left, elements  $\geq 2$  on the right.
  - Elements  $\leq 2$ : [1]
  - Elements  $\geq 2$ : [4, 3]

The first sub-list is further divided:

[1]    2    [4, 3]

(Sub-list 1A) (Pivot) (Sub-list 1B)

[1]    2    [4, 3]

## 3. Further Recursion (Conquer)

This process continues recursively until all sub-lists are trivially sorted (i.e., contain only one element)

- **Sub-list 1A: [1]**
  - **Base Case:** Contains 1 element. It is sorted.
- **Sub-list 1B: [4, 3]**
  - **Choose Pivot:** Select **3**.
  - **Partition:** Elements  $\leq 3$ : [] / Elements  $\geq 3$ : [4]
  - Result: 3 [4]. Now sort [4].
  - **Base Case:** [4] is sorted.
- **Sub-list 2: [9, 7, 10, 6]**
  - A full trace would break this down further, for example, choosing **6** as the pivot initially places 6 correctly, and then the remaining unsorted section becomes [9, 7, 10].
  - ...The process continues until all elements are positioned correctly.

The final result after all recursive calls return is the complete sorted list:

[1, 2, 3, 4, 5, 6, 7, 9, 10]

This hierarchical breaking down of the problem demonstrates the **recursive nature** of quicksort.

## Quicksort Code

```
def quick_sort(arr):  
    """  
    The recursive Quicksort function.  
  
    Args:  
        arr: The list to be sorted.  
  
    Returns:  
        A new sorted list.  
    """  
    if len(arr) < 2:  
        # Base case: A list with 0 or 1 element is already sorted [cite: 4016]  
        return arr  
    else:  
        # Recursive case  
        pivot = arr[0] # Choose the first element as the pivot [cite: 4031]  
  
        # Elements less than or equal to the pivot (excluding the pivot itself)  
        less = [i for i in arr[1:] if i <= pivot]  
  
        # Elements greater than the pivot [cite: 4034]  
        greater = [i for i in arr[1:] if i > pivot]  
  
        # Recursively sort the sub-lists and combine them with the pivot [cite: 4035]  
        return quick_sort(less) + [pivot] + quick_sort(greater)
```

**Merge Sort:** Another divide-and-conquer sorting algorithm that recursively splits the array, sorts the sub-arrays, and merges them.

**Tower of Hanoi:** A classic puzzle that involves moving disks between pegs, which can be solved elegantly using a recursive algorithm.

**Tree Traversal:** Recursion is naturally suited for traversing tree structures like binary trees. Common tree traversal algorithms, such as inorder, preorder, and postorder traversal, are implemented recursively.

**Fractal Generation:** Fractals are self-similar patterns that can be generated using recursive algorithms. Examples include the Mandelbrot set and the Sierpinski triangle.

## Limitations of Recursion

**Stack Overflow:** As mentioned earlier, deep recursion can lead to stack overflow errors. This is a major limitation, especially when dealing with problems requiring many recursive calls.

**Performance:** Recursive function calls can introduce overhead compared to iterative solutions. This overhead can be significant for deep recursion or problems with simple recursive structures that can be easily converted to loops.

**Memory Usage:** Recursion uses stack memory to store information about each function call, including local variables, parameters, and the return address. Excessive recursion can lead to high memory usage, potentially exceeding the available stack space.

## When to Use Recursion

**Suitable Problems:** Recursion is ideal for problems that can be naturally broken down into smaller, self-similar subproblems. This often involves a divide-and-conquer approach.

**Code Readability:** If code readability and maintainability are priorities, and performance is not a critical concern, recursion can be a good choice.

## Key Questions

**Hinge Question:** Explain how the base and recursive cases work together to ensure that a recursive function terminates.

**Answer:** The base case provides a condition for stopping the recursion. The recursive case makes a recursive call with a smaller subproblem, eventually leading to the base case. Without a base case, the recursion would continue infinitely.

**Hinge Question:** Describe a scenario where recursion might be less efficient than an iterative solution.

Answer: Recursion can be less efficient when the recursive structure is simple and easily converted to a loop. The overhead of function calls in recursion can add up, especially for deep recursion. For example, calculating the sum of a list might be more efficiently done with a loop than with recursion.

Hinge Question: Why is it important to be mindful of the potential for stack overflow errors when using recursion?

Answer: Stack overflow errors occur when the call stack exceeds its capacity due to excessive recursion. This can lead to program crashes and unexpected behaviour. Therefore, it's crucial to design recursive algorithms carefully, considering the depth of recursion and the size of the input data.

---

*All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.*

---

 **Report a problem**